Multi-Stock Prediction for Medium-Term Financial Forecasting

Brendan Ng and Ben Fitzgerald

Overview

Can deep learning improve medium-term stock return prediction beyond simple baseline?

Stocks:

- AAPL
- MSFT
- NVDA
- GOOGL
- META
- TSLA

60 day lookback | 30 day forecast

Objective:

We trained LSTM, MLP, Prophet, and Transformer models in PyTorch to forecast 30-day log returns for 7 tech stocks, compared them against a persistence baseline, and discovered that rigorous leakage prevention matters more than model complexity in financial ML.

Models and Evaluation

- Persistence Baseline
  - Assumes future return = last observed return
- LSTM
  - A type of neural network that reads data in order, like reading a sentence word by word, and remembers what came before.
- MLP
  - A simpler neural network that looks at all the information at once without caring about order.
- Transformer
  - Causal transformer designed for time-series forecasting in financial data

Core Equations

- Log return: $r_t = \ln(P_{t+30} / P_t)$
- MSE loss: $L = (1/N) \Sigma (\hat{y}_i - y_i)^2$
- LSTM:
  - $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ ← forget gate
  - $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ ← input gate
  - $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$ ← candidate cell
  - $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ ← cell state update
  - $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$ ← output gate
  - $h_t = o_t \odot \tanh(C_t)$ ← hidden state output
- MLP forward pass equations
  - $h_1 = \text{ReLU}(W_1 \cdot x + b_1)$
  - $h_2 = \text{ReLU}(W_2 \cdot h_1 + b_2)$
  - $\hat{y} = W_{out} \cdot h_2 + b_{out}$
- Transformer
  - $\text{Attention}(Q, K, V) = \text{softmax}((Q \times K^T) / \sqrt{d_k}) \times V$
  - $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}\square) \times W^o$
  - $\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$
  - $y = x + \text{sigmoid}(W[x ; f(x)]) \times f(x)$
- RSI, MA, volatility formulas
  - MA_5 = average closing price over last 5 days
  - MA_20 = average closing price over last 20 days
  - Volatility = std deviation of log returns over last 20 days
  - RSI = 100 – 100 / (1 + avg_gain / avg_loss) [14-day window]

Data & Tensor Pipeline

Step by step

- Raw input: daily close prices from yfinance
- Log return computation: r_t = ln(P_{t+30} / P_t)
- Technical indicator computation
- Setting the target(30-day forward offset)
- Sliding window
- Tensor shapes:

```
X (returns only):      shape = (N, seq_len, 7)
X (with indicators):   shape = (N, seq_len, 35)
y (target):            shape = (N, 7)
```

- Chronological train/test split
- Batching with DataLoader

```
x_batch : (batch_size, seq_len, F)
y_batch : (batch_size, 7)
```

Architecture

- LSTM: input shape → recurrent layers → final hidden state h_T → linear head → 7-dim output
- MLP: input → flatten → fully connected layers → 7-dim output
- Transformer: input sequence → embedding layer → positional encoding → stacked multi-head self-attention + feed-forward blocks → sequence aggregation (e.g., CLS token or pooling) → linear head → 7-dim output
- Task type: multi-output regression (continuous log return values, not classification)
- Output representation: 7 predicted 30-day log returns, one per stock
- Weight initialization: PyTorch defaults (Kaiming uniform for linear, uniform scaled by 1/√hidden for LSTM), trained from scratch, no pretrained backbone

Results

- 7 experimental conditions: Persistence Baseline, LSTM × 2, MLP × 2, Transformer x 2
- Feature sets: returns-only vs. returns + technical indicators (MA5, MA20, volatility, RSI)
- Metrics tracked: Test MSE, Directional Accuracy, Total Trading Return, Sharpe Ratio
- Best model: LSTM + indicators (MSE 0.0398, Sharpe −0.13)
- All neural models produced trading return of −1.00 after transaction costs
- Logging: epoch-by-epoch metrics via custom CSV logger (history.csv, metrics.csv)
- Backtesting: long/short strategy with equal weighting across 7 stocks, transaction cost penalty applied

TABLE I
FINAL MODEL PERFORMANCE SUMMARY (LEAKAGE-FREE)

Model	Features	Test Loss	Dir. Acc.	Return	Sharpe
Persistence	Returns	0.0209	0.521	N/A	N/A
LSTM	Returns	0.0543	0.363	-1.00	-0.19
LSTM	+Indicators	0.0398	0.393	-1.00	-0.13
MLP	Returns	0.0844	0.318	-1.00	-0.40
MLP	+Indicators	0.0933	0.347	-1.00	-0.31
Transformer	Returns	0.0605	0.367	-1.00	-0.35
Transformer	+ Indicators	0.0490	0.348	-1.00	-0.31

Data Leakage Bugs

- Misaligned Target Construction
- Feature leakage from technical indicators
- Persistence baseline leakage(indexing bug)
- Train/Test Contamination
- Backtesting inflation

After Fixes

- Sharpe ratios dropped
- Directional accuracy changes
- LSTM showed some improvement
- Persistence baseline became a strong benchmark

Key Findings

- LSTM with technical indicators is the best model on every metric tracked.
- No model produced profitable out-of-sample trading returns.
- The persistence baseline outperforms all neural models on directional accuracy.
- The primary contribution of this project is the leakage-free evaluation pipeline.